REACT WEB

React 2 — Counter: полное руководство

Пошаговое учебное руководство
от условия до проверки и сдачи

Обязательная работа + Bonus

Учебный комплект • 2026

> Статус: обязательная работа; Bonus «не опускаться ниже нуля» выполнен. Готовое решение находится в `projects/react-web/02-counter`.

Что требуется

На странице постоянно видно текущее значение счётчика и три кнопки:

- Increase увеличивает значение на один;
- Decrease уменьшает значение на один;
- Reset возвращает значение к нулю;
- Bonus: Decrease не позволяет получить отрицательное число.

Что получится

Итог — веб-приложение React + TypeScript + Vite. React хранит число в state и перерисовывает интерфейс после обновления. Отдельная чистая функция вычисляет следующее значение, поэтому одни и те же правила используются компонентом и модульными тестами.

Как отличить правильный проект

В корне правильного Vite-проекта находятся `package.json`, `index.html`, `vite.config.ts` и каталог `src`. Весь код приложения расположен внутри `src`; браузерная точка входа — `src/main.tsx`.

Не создавайте `App.tsx` или `App.js` в корне. Не добавляйте `app.json`, Expo, `react-native`, `View`, `Text` или `StyleSheet`. В этой работе используются браузерные элементы `main`, `output`, `div`, `button` и обычный CSS.

Подготовка на macOS

Поддерживается Node.js `^20.19.4 || >=22.12.0`:

- Node 20 подходит начиная с 20.19.4 и до перехода на Node 21;
- Node 21 не подходит;
- Node 22 подходит начиная с 22.12.0;
- более новые версии подходят по диапазону проекта.

Проверьте инструменты:

КОД · BASH

```
node --version
npm --version
git --version
```

Если Vite сообщает о неподдерживаемом Node.js, обновите Node.js. Не удаляйте lock-файл ради обхода проверки.

Создание с нуля

1. Откройте приглашение GitHub Classroom и примите задание.
2. Дождитесь создания персонального репозитория.

3. Клонируйте именно этот репозиторий и перейдите в его корень:

КОД · BASH

```
git clone <адрес-вашего-classroom-репозитория>
cd <имя-репозитория>
```

4. Проверьте папку:

КОД · BASH

```
git status
ls
```

Рядом должны находиться `package.json` и `src`. Если видны Expo-файлы или корневой `App.tsx`, открыт неправильный шаблон.

5. В готовом Classroom-репозитории сохраняйте конфигурацию преподавателя. Не заменяйте `package.json`, lock-файл, Vite- или TypeScript-конфигурацию файлами из другого проекта. Создавайте и изменяйте только файлы внутри `src`.
6. Восстановите зафиксированные зависимости:

КОД · BASH

```
npm ci
```

Чем отличаются npm install и npm ci

`npm install` выбирает версии по диапазонам из `package.json` и может изменить `package-lock.json`. Команда нужна при первом создании собственного проекта или при осознанном добавлении зависимости.

`npm ci` требует готовый `package-lock.json`, пересоздаёт `node_modules` и устанавливает точное дерево зависимостей. После клонирования Classroom-репозитория и перед сдачей используйте `npm ci`, чтобы окружение совпадало с окружением проверки.

Структура файлов

КОД · TEXT

```
02-counter/
├── index.html
├── package.json
├── package-lock.json
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.node.json
├── vite.config.ts
└── src/
    ├── App.css
    ├── App.test.tsx
    ├── App.tsx
    ├── counter.test.ts
    ├── counter.ts
    ├── main.tsx
    ├── vite-env.d.ts
    └── test/
        └── setup.ts
```

`counter.ts` отвечает только за правила перехода числа. `App.tsx` хранит состояние и связывает эти правила с кнопками. Такое разделение позволяет проверить вычисления без браузерного интерфейса.

Пошаговая реализация

Каждый листинг относится к отдельному файлу по указанному пути. Не вставляйте все блоки в один `App.tsx`.

src/counter.ts

КОД · TS

```ts
export const COUNTER_ACTIONS = {
  INCREASE: 'increase',
  DECREASE: 'decrease',
  RESET: 'reset',
} as const;

export type CounterAction =
  (typeof COUNTER_ACTIONS)[keyof typeof COUNTER_ACTIONS];

// Ограничение в одном переходе делает бонус одинаковым для клика и теста.
export function nextCounterValue(
  value: number,
  action: CounterAction,
): number {
  switch (action) {
    case COUNTER_ACTIONS.INCREASE:
      return value + 1;
    case COUNTER_ACTIONS.DECREASE:
      return Math.max(0, value - 1);
    case COUNTER_ACTIONS.RESET:
      return 0;
    default:
      return value;
  }
}
```

Что используется и зачем:

- `const COUNTER_ACTIONS` создаёт единственный словарь допустимых действий. Компонент и тесты не дублируют строковые литералы.
- `export` разрешает импорт объекта, типа и функции из других модулей.
- `as const` сохраняет точные литеральные значения `'increase'`, `'decrease'`, `'reset'` и делает свойства объекта readonly на уровне типов.
- `typeof COUNTER_ACTIONS` получает TypeScript-тип объекта, а `keyof` получает объединение его ключей.
- Индексированный доступ `(typeof COUNTER_ACTIONS)[keyof typeof COUNTER_ACTIONS]` превращает значения объекта в union-тип `'increase' | 'decrease' | 'reset'`.
- `nextCounterValue(value: number, action: CounterAction): number` принимает текущее число и разрешённое действие, затем гарантированно возвращает число.
- Функция чистая: она не меняет React state, DOM или внешние переменные. Для одинаковых аргументов результат одинаков.
- `switch (action)` выбирает одну ветвь. Каждый `case` сравнивает действие с константой и сразу возвращает результат.

- Increase возвращает `value + 1`.
- Decrease использует `Math.max(0, value - 1)`: из нуля получается максимум из 0 и -1, то есть 0. Так Bonus реализован в общем правиле, а не только в кнопке.
- Reset возвращает ровно 0 независимо от текущего значения.
- `default` сохраняет значение при некорректном JavaScript-вызове. TypeScript не разрешает такой action в типизированном коде, но runtime-защита остаётся полезной.

src/App.tsx

КОД · TSX

```tsx
import { useState } from 'react';
import './App.css';
import {
  COUNTER_ACTIONS,
  nextCounterValue,
  type CounterAction,
} from './counter';

export default function App() {
  const [value, setValue] = useState<number>(0);

  function dispatch(action: CounterAction): void {
    setValue((currentValue) => nextCounterValue(currentValue, action));
  }

  return (
    <main className="page">
      <h1>Counter</h1>
      <output className="counter-value" aria-live="polite">
        Current value: {value}
      </output>
      <div className="controls" role="group" aria-label="Counter controls">
        <button type="button" onClick={() => dispatch(COUNTER_ACTIONS.INCREASE)}>Increase</button>
        <button type="button" onClick={() => dispatch(COUNTER_ACTIONS.DECREASE)}>Decrease</button>
        <button type="button" onClick={() => dispatch(COUNTER_ACTIONS.RESET)}>Reset</button>
      </div>
    </main>
  );
}
```

Что используется и зачем:

- `useState` — React hook для локального состояния функционального компонента.
- `import './App.css'` подключает веб-CSS через Vite.
- `type CounterAction` импортируется с ключевым словом `type`, потому что нужен только TypeScript и не существует в готовом JavaScript.
- `useState<number>(0)` создаёт числовое состояние с начальным значением 0. Generic `<number>` явно фиксирует допустимый тип.
- Деструктуризация массива `[value, setValue]` даёт текущее значение и setter-функцию.
- `dispatch(action: CounterAction): void` принимает только одно из трёх допустимых действий и ничего не возвращает.
- `setValue((currentValue) => ...)` — функциональное обновление. React передаёт callback самое актуальное состояние, поэтому быстрые последовательные клики не теряются из-за значения старого рендера.

- Callback вызывает чистую `nextCounterValue`, а React сохраняет её результат и выполняет новый рендер.
- `output` семантически обозначает результат вычисления. `aria-live="polite"` просит экранный диктор сообщать изменение без резкого прерывания пользователя.
- `role="group"` и `aria-label="Counter controls"` объединяют кнопки в доступную группу.
- `type="button"` не позволяет кнопке случайно отправлять форму, если компонент позже окажется внутри `form`.
- `onClick` получает стрелочную функцию, а не результат `dispatch`. Поэтому действие выполняется только после клика.
- Для каждой кнопки передаётся своя константа из `COUNTER_ACTIONS`.

src/App.css

КОД · CSS

```
:root { font-family: Arial, sans-serif; color: #1f2937; background: #f8fafc; }
body { margin: 0; }
.page { max-width: 480px; margin: 0 auto; padding: 32px 16px; text-align: center; }
.counter-value { display: block; margin: 24px 0; font-size: 2rem; font-weight: 700; }
.controls { display: flex; flex-wrap: wrap; justify-content: center; gap: 12px; }
button { min-width: 100px; padding: 10px 14px; border: 1px solid #1d4ed8; border-radius: 6px;
background: #2563eb; color: #ffffff; font: inherit; cursor: pointer; }
button:focus-visible { outline: 3px solid #92400e; outline-offset: 2px; }
```

Что используется и зачем:

- `:root` задаёт общие шрифт, цвет текста и фон.
- `body { margin: 0; }` удаляет стандартный отступ браузера.
- `.page` ограничивает ширину, центрирует приложение и выравнивает текст.
- `.counter-value` превращает `output` в блочный элемент, отделяет его и увеличивает размер числа.
- `.controls` использует Flexbox. `flex-wrap: wrap` переносит кнопки на узком экране, `justify-content: center` центрирует их, `gap` задаёт расстояние.
- `button` получает минимальную ширину, отступы, рамку, скругление, цвета, наследуемый шрифт и курсор.
- `button:focus-visible` показывает заметный контур при клавиатурной навигации, а `outline-offset` отделяет его от кнопки.
- Это обычный CSS. React Native `StyleSheet` здесь не используется.

src/main.tsx

КОД · TSX

```tsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';

const rootElement = document.getElementById('root');

if (rootElement === null) {
  throw new Error('Root element #root was not found.');
}

createRoot(rootElement).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

Что используется и зачем:

- `StrictMode` включает дополнительные проверки React во время разработки.
- `createRoot` из `react-dom/client` подключает React к браузерному DOM.
- `document.getElementById('root')` находит контейнер из корневого `index.html`.
- Проверка на `null` даёт понятную runtime-ошибку и сужает тип для TypeScript.
- Метод `render` создаёт `App` внутри `#root`.
- Этот файл находится в `src`; корневой `App.tsx` для Vite не создаётся.

src/test/setup.ts

КОД · TS

```ts
import '@testing-library/jest-dom/vitest';
```

Что используется и зачем:

- Импорт выполняется ради побочного эффекта и расширяет `expect` DOM-матчерами.
- Благодаря ему тесты используют `toHaveTextContent`, `toContainElement` и другие проверки интерфейса.
- Готовый `vite.config.ts` запускает setup перед тестовыми файлами.

src/counter.test.ts

КОД · TS

```ts
import { COUNTER_ACTIONS, nextCounterValue } from './counter';

test('increases the current value by one', () => {
  expect(nextCounterValue(4, COUNTER_ACTIONS.INCREASE)).toBe(5);
});

test('decreases a positive value by one', () => {
  expect(nextCounterValue(4, COUNTER_ACTIONS.DECREASE)).toBe(3);
});

test('implements the bonus by keeping zero at zero when decreasing', () => {
  expect(nextCounterValue(0, COUNTER_ACTIONS.DECREASE)).toBe(0);
});

test('resets any value to zero and ignores an unknown action', () => {
  expect(nextCounterValue(17, COUNTER_ACTIONS.RESET)).toBe(0);
  // @ts-expect-error Проверяем защиту от некорректного JavaScript-вызова во время выполнения.
  expect(nextCounterValue(17, 'unknown')).toBe(17);
});
```

Что используется и зачем:

- Каждый `test` проверяет одно правило перехода и получает понятное текстовое имя.
- `expect(actual).toBe(expected)` использует строгое сравнение чисел.
- Тесты Increase и Decrease начинают с 4, чтобы различать прибавление и вычитание.
- Отдельный тест нуля закрепляет Bonus и не позволяет случайно вернуть `-1`.
- Reset проверяется с ненулевого 17, поэтому тест действительно обнаружит отсутствие сброса.
- `@ts-expect-error` подтверждает, что TypeScript запрещает `'unknown'`, но позволяет намеренно проверить ветку runtime-защиты `default`.
- Здесь не нужен React render: чистая функция проверяется как обычная TypeScript-функция.

src/App.test.tsx

КОД · TSX

```tsx
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import App from './App';

test('uses useState-driven buttons and never displays a negative value', async () => {
  const user = userEvent.setup();
  render(<App />);

  const value = screen.getByRole('status');
  expect(value).toHaveTextContent('Current value: 0');

  await user.click(screen.getByRole('button', { name: 'Increase' }));
  await user.click(screen.getByRole('button', { name: 'Increase' }));
  expect(value).toHaveTextContent('Current value: 2');

  await user.click(screen.getByRole('button', { name: 'Decrease' }));
  await user.click(screen.getByRole('button', { name: 'Decrease' }));
  await user.click(screen.getByRole('button', { name: 'Decrease' }));
  expect(value).toHaveTextContent('Current value: 0');

  await user.click(screen.getByRole('button', { name: 'Increase' }));
  await user.click(screen.getByRole('button', { name: 'Reset' }));
  expect(value).toHaveTextContent('Current value: 0');
```

```
});

test('groups the counter controls under an accessible label', () => {
  render(<App />);

  const controls = screen.getByRole('group', { name: 'Counter controls' });
  expect(controls).toContainElement(screen.getByRole('button', { name: 'Increase' }));
  expect(controls).toContainElement(screen.getByRole('button', { name: 'Decrease' }));
  expect(controls).toContainElement(screen.getByRole('button', { name: 'Reset' }));
});
```

Что используется и зачем:

- `render(<App />)` создаёт весь компонент в JSDOM.
- `screen` ищет элементы по доступным ролям и именам, а не по внутренним CSS-классам.
- `output` с `aria-live` имеет роль `status`, поэтому `getByRole('status')` получает видимое значение.
- `userEvent.setup()` создаёт объект пользователя. Его `click` возвращает Promise, поэтому тест объявлен `async`, а каждый клик ожидается через `await`.
- Два Increase закрепляют последовательное функциональное обновление до 2.
- Три Decrease после 2 проверяют и обычное уменьшение, и нижнюю границу 0.
- Increase и Reset подтверждают сброс после ненулевого состояния.
- Второй тест получает группу по роли и имени, затем `toContainElement` проверяет принадлежность каждой кнопки группе.

src/vite-env.d.ts

КОД · TS

```
/// <reference types="vite/client" />
```

Что используется и зачем:

- Тройная директива подключает клиентские типы Vite.
- TypeScript получает определения возможностей Vite, включая `import.meta.env`.
- В файле нет runtime-кода; шаблон `react-ts` обычно создаёт его заранее.

Порядок реализации от пустого решения до готового

1. После `npm ci` выполните исходный тестовый запуск:

КОД · BASH

```
npm run test:run
```

2. Создайте `src/counter.test.ts` с примерами переходов. До создания функции тест ожидаемо завершится ошибкой.
3. Создайте `src/counter.ts` и повторяйте только модульный тест, пока все ветви не заработают:

КОД · BASH

```
npm run test:run -- src/counter.test.ts
```

4. Создайте `src/App.test.tsx` и полностью замените `src/App.tsx`.

5. Замените `src/App.css` и сверьте `src/main.tsx`, `src/test/setup.ts`, `src/vite-env.d.ts` с полными листингами.
6. Выполните весь набор тестов и исправляйте первую фактическую ошибку, не отключая строгие TypeScript-настройки.

Как проходит один клик

Для Increase цепочка выглядит так:

1. Пользователь нажимает кнопку.
2. Стрелочная функция вызывает `dispatch(COUNTER_ACTIONS.INCREASE)`.
3. `dispatch` вызывает setter `setValue` в функциональной форме.
4. React передаёт callback актуальный `currentValue`.
5. `nextCounterValue` выбирает ветку `switch` и возвращает `currentValue + 1`.
6. React сохраняет новое число и повторно вызывает `App`.
7. JSX вставляет обновлённый `value` в `output`.

Decrease и Reset проходят ту же цепочку, меняется только ветвь чистой функции.

Запуск и тесты

Из корня проекта выполните:

КОД · BASH

```
npm run typecheck
npm run test:run
npm run build
```

- `typecheck` запускает `tsc -b` и проверяет state, действия, imports и JSX без создания JavaScript рядом с исходниками.
- `test:run` запускает Vitest один раз; готовое решение содержит шесть сценариев в двух тестовых файлах.
- `build` повторяет TypeScript-проверку и создаёт production-каталог `dist`.

Для автоматического повтора тестов после сохранения:

КОД · BASH

```
npm test
```

Не переходите к ручной проверке, пока первая из gate-команд завершается ошибкой.

Dev server и Preview

Запустите режим разработки:

КОД · BASH

```
npm run dev
```

Vite обычно сообщает адрес `http://localhost:5173`. Здесь работает Hot Module Replacement: изменения в `src` быстро появляются без полной перезагрузки. Остановите сервер сочетанием Control+C.

После `npm run build` проверьте собранный `dist`:

КОД · BASH

```
npm exec -- vite preview
```

Vite Preview обычно открывается на `http://localhost:4173`. Dev server на 5173 предназначен для разработки; Preview на 4173 раздаёт production-сборку и запускается только после build.

Ручная проверка

1. После загрузки видно `Current value: 0`.
2. Нажмите Increase три раза: должно стать 3.
3. Нажмите Decrease один раз: должно стать 2.
4. Нажмите Reset: должно стать 0.
5. Нажмите Decrease пять раз: значение должно остаться 0.
6. Быстро нажмите Increase несколько раз: каждый клик должен учитываться.
7. Перейдите между кнопками клавишей Tab: видимый контур фокуса должен сохраняться.
8. Обновите страницу: сохранение состояния не требовалось, поэтому допустим сброс к 0.
9. Убедитесь, что в консоли браузера нет runtime-ошибок.

Отправка в GitHub Classroom

Проверьте изменения:

КОД · BASH

```
git status
git diff -- src
```

Добавьте только выполненные исходники:

КОД · BASH

```
git add src
git commit -m "Выполнить задание Counter"
git push
```

Не отправляйте `node_modules`, `dist`, `.env`, Expo-файлы или корневой `App.tsx`. После push откройте GitHub и убедитесь, что последний коммит появился в персональном Classroom-репозитории.

Типичные ошибки

- `let value = 0` внутри компонента не является state: после рендера такая переменная создаётся заново и не управляет интерфейсом.
- `setValue(value + 1)` использует значение из конкретного рендера; функциональный setter безопаснее для последовательных обновлений.
- `onClick={dispatch(...)}` вызывает функцию во время рендера. Нужен callback `onClick={() => dispatch(...)}`.
- Прямое `value - 1` даёт -1 на нуле. `Math.max(0, value - 1)` устанавливает нижнюю границу.

- Проверка `if (value > 0)` только в UI дублирует правило и оставляет чистую функцию без защиты.
- Без `as const` значения объекта могут расшириться до общего `string`, и точный `CounterAction` потеряется.
- `dispatch(action: string)` разрешает опечатки; параметр должен иметь тип `CounterAction`.
- Удаление `@ts-expect-error` при сохранении вызова с `'unknown'` закономерно ломает typecheck.
- Корневой `App.tsx`, `StyleSheet`, `View` или Expo означают смешение веб- и мобильного шаблонов.
- Reset должен сразу вернуть 0, а не вычитать текущее значение несколькими действиями.

Вопросы и ответы

Почему состояние хранится в `App`? Значение определяет видимый интерфейс и меняется после событий, поэтому React state является владельцем числа и запускает перерисовку.

Зачем логика вынесена в `counter.ts`? Чистую функцию можно проверить без DOM, а компонент использует то же правило для реальных кликов.

Почему нужен функциональный setter? Callback получает актуальное значение от React и не зависит от замыкания конкретного рендера.

Почему Decrease не отключается на нуле? Bonus реализован в общем переходе `Math.max(0, value - 1)`, поэтому правило одинаково для интерфейса и тестов.

Чем Preview отличается от dev server? Dev server на 5173 работает с исходниками и HMR, а Preview на 4173 раздаёт production-сборку из `dist`.

Защита

1. Что хранит `useState<number>(0)` и что возвращает hook.
2. Почему setter вызывается с функцией `(currentValue) => ...`.
3. Почему `nextCounterValue` считается чистой функцией.
4. Как `as const`, `typeof`, `keyof` и индексированный доступ образуют `CounterAction`.
5. Как `switch` связывает три действия с тремя переходами.
6. Почему `Math.max(0, value - 1)` выполняет Bonus.
7. Чем модульный тест `counter.ts` отличается от компонентного теста с кликами.
8. Чем отличаются typecheck, test, build, dev server и Preview.

Будьте готовы изменить шаг Increase с одного на два: меняется ветвь Increase чистой функции и соответствующие ожидаемые значения тестов.

Чек-лист сдачи

- ☐ Открыт React + TypeScript + Vite, а не Expo.
- ☐ Все изменённые исходники находятся внутри `src`.
- ☐ Начальное значение равно 0 и постоянно видно.
- ☐ Increase прибавляет один.
- ☐ Decrease вычитает один до нижней границы 0.

- ☐ Reset возвращает ровно 0.
- ☐ `CounterAction` допускает только три действия.
- ☐ `nextCounterValue` остаётся чистой функцией.
- ☐ `App` использует `useState<number>` и функциональный setter.
- ☐ Модульные и компонентные тесты присутствуют.
- ☐ `npm run typecheck` завершён без ошибок.
- ☐ `npm run test:run` завершён без ошибок.
- ☐ `npm run build` создал `dist`.
- ☐ Dev server проверен на порту 5173, Preview — на порту 4173.
- ☐ В commit не попали `node_modules`, `dist`, корневой `App.tsx` и Expo-файлы.